Q1: What format specifier does printf use for a double, and what specifier does scanf use to read into a pointer to double? Why do they differ?

printf uses %f to print a double. scanf uses %lf to read a double. They are different because printf automatically changes a float to a double, but scanf needs %lf to know that it is reading a double.

Q2: By hand, compute 7 / 2 and 7.0 / 2 in C and explain why the two results are different.

7 / 2 = 3

7.0 / 2 = 3.5

7 / 2 gives 3 because they are both integers, so C drops the decimal part. 7.0 / 2 gives 3.5 because 7.0 is a decimal number, so C keeps the decimal in the answer.

Q3: What does scanf return when it successfully matches all requested conversions? What does it return if the very first conversion fails?

scanf returns how many values it was able to read. For example, if it reads one value correctly, it returns 1.

If you type something that is not the right type, it returns 0. It can also return EOF if there is no more input.

Q4: If a program calls scanf("%d", &x) and the user types 'abc', what happens to x, and what is left in the input stream?

x stays the same because scanf could not read the input correctly.

The abc is still in the input. If you try to use scanf again without getting rid of it, it can keep trying to read the same abc over and over.

Q5: Why should a conversion function be written to take and return double rather than int, even if the values involved are often whole numbers?

Using double is good because it can handle numbers with decimals. It also helps keep the answer more accurate when doing calculations like division.